



Promises

A espera do que não vemos

- ☐ Você já foi a um concerto?

Básico

```
let promise = new Promise(function (resolve, reject) {  
  // executor (o código que produz, "cantor")  
});
```

- ☐ A função "executora"
- ☐ `resolve` e `reject`
- ☐ Estados: `pending`, `fulfilled` e `rejected`
- ☐ Consumindo Promises
 - ☐ `then`, `catch` e `finally`

Encadeamento de *Promises*

- ☐ *Handlers* das Promises
- ☐ O que é e não é considerado encadeamento
- ☐ Retornando Promise a partir do `then`
- ☐ Encadeando requests

Lidando com erros

- ☐ `catch` como fallback
- ☐ O `try..catch` implícito
- ☐ Criando novos erros e encadeando o `catch`

API das *Promises*

- ☐ `all()`
- ☐ `allSettled()`
- ☐ `race()`
- ☐ `any()`

Async/Await

- ☐ Uma alternativa para lidar com Promises.
- ☐ Erros no `await`

The Modern JavaScript Tutorial

Modern JavaScript Tutorial: simple, but detailed explanations with examples and tasks, including: closures, document and events, object oriented programming and

 <https://javascript.info/>



JAVASCRIPT.INFO
The Modern JavaScript Tutorial

Exercícios

Promises Básicas

1. Crie uma Promise que resolva com o valor `"Olá mundo"` após 2 segundos.
2. Crie uma Promise que rejeite com o erro `"Algo deu errado"` após 1 segundo.
3. Utilize `.then()` e `.catch()` para consumir uma Promise que resolve com o número `10`. Multiplique o valor por 5.
4. Crie uma função que retorna uma Promise que resolve com um número aleatório entre 1 e 100 após 1 segundo.
5. Encadeie 3 `.then()` para transformar o número: dobre, adicione 10, divida por 2.
6. Crie uma função que recebe um nome e retorna uma Promise que resolve com `"Olá, {nome}"` após 1 segundo.
7. Simule um erro proposital na segunda etapa de um `.then()`. Use `catch()` para capturá-lo.
8. Use `finally()` para exibir `"Operação concluída"` independente do sucesso ou erro de uma Promise.
9. Use uma função que retorna uma Promise com base em um `setTimeout()` e simule um "carregando..." antes de resolver.
10. Crie uma Promise que verifica se um número é par. Resolva se for par, rejeite se for ímpar.

Fetch com `then` e `catch`

11. Use `fetch()` para buscar `https://dummyjson.com/products/1`. Exiba o nome do produto no console.
12. Crie uma função `getProduct(id)` que retorna uma Promise com os dados de um produto específico da API.
13. Faça 3 requisições em sequência (produto 1, 2 e 3), usando encadeamento de `.then()`.
14. Tente buscar um produto que não existe (ex: `id = 9999`) e trate o erro com `.catch()`.
15. Use `fetch()` para buscar `https://dummyjson.com/users/1` e exibir nome e e-mail.
16. Crie uma função que recebe um `userId` e retorna o nome completo do usuário (`firstName lastName`).

17. Use o método `.text()` da resposta do fetch e mostre o conteúdo cru da resposta.
 18. Tente usar `.json()` duas vezes na mesma resposta. O que acontece? Teste e explique no código.
 19. Encadeie dois fetchs: um para `products/1` e o segundo para `users/1`, mostrando o nome do produto e o usuário.
 20. Simule uma falha no `fetch` alterando a URL para `https://dummyjson.com/erro`. Trate com `catch()`.
-

Async/Await

21. Refaça o exercício 11 usando `async/await`.
22. Crie uma função `getUserNome(id)` que usa `await fetch` para retornar o nome do usuário.
23. Use `try/catch` com `await fetch` para lidar com um erro de requisição (ex: URL incorreta).
24. Faça uma função `getProductPrice(id)` que retorna apenas o preço de um produto.
25. Crie uma função que retorna um array de títulos de 3 produtos diferentes, usando `await` em cada chamada.
26. Use `await` dentro de um loop `for` para buscar os produtos de id 1 a 3, um por vez.
27. Crie uma função `loadProduct(id)` que mostra "Carregando..." no console, depois o produto.
28. Crie uma função `getUserAvatar(id)` que busca a foto de um usuário (`image`) e a exibe no DOM (HTML).
29. Crie uma função que faça uma requisição com `await fetch`, mas jogue um erro proposital com `throw new Error`.
30. Use `await fetch` para obter um usuário. Se a idade for menor que 30, jogue erro. Trate com `try/catch`.

```
// PROMISES BÁSICAS (1-10)
```

```
// 1.
```

```
new Promise(resolve => setTimeout(() => resolve("Olá mundo"), 2000));
```

```
// 2.
```

```
new Promise((_, reject) => setTimeout(() => reject("Algo deu errado"), 1000));
```

```
// 3.
```

```
Promise.resolve(10)
  .then(num => num * 5)
  .then(console.log)
  .catch(console.error);
```

```
// 4.
```

```
function randomNumber() {
  return new Promise(resolve => setTimeout(() => resolve(Math.floor(Math.random() * 100) + 1), 1000));
}
```

```
// 5.
```

```
Promise.resolve(5)
  .then(n => n * 2)
  .then(n => n + 10)
  .then(n => n / 2)
  .then(console.log);
```

```
// 6.
```

```
function greet(name) {
  return new Promise(resolve => setTimeout(() => resolve(`Olá, ${name}`), 1000));
}
```

```
// 7.
```

```
Promise.resolve(10)
  .then(n => n * 2)
  .then(() => { throw new Error("Erro proposital"); })
```

```

    .then(console.log)
    .catch(console.error);

// 8.
Promise.resolve("OK")
    .then(console.log)
    .catch(console.error)
    .finally(() => console.log("Operação concluída"));

// 9.
function delayedMessage() {
    console.log("Carregando...");
    return new Promise(resolve => setTimeout(() => resolve("Pronto!"), 1000));
}

// 10.
function isEven(num) {
    return new Promise((resolve, reject) => {
        num % 2 === 0 ? resolve("Par") : reject("Ímpar");
    });
}

// FETCH COM THEN/CATCH (11-20)

// 11.
fetch("https://dummyjson.com/products/1")
    .then(res => res.json())
    .then(data => console.log(data.title));

// 12.
function getProduct(id) {
    return fetch(`https://dummyjson.com/products/${id}`).then(res => res.json());
}

// 13.
fetch("https://dummyjson.com/products/1")
    .then(res => res.json())

```

```

.then(data => {
  console.log(data.title);
  return fetch("https://dummyjson.com/products/2");
})
.then(res => res.json())
.then(data => {
  console.log(data.title);
  return fetch("https://dummyjson.com/products/3");
})
.then(res => res.json())
.then(data => console.log(data.title));

// 14.
fetch("https://dummyjson.com/products/9999")
.then(res => {
  if (!res.ok) throw new Error("Produto não encontrado");
  return res.json();
})
.catch(console.error);

// 15.
fetch("https://dummyjson.com/users/1")
.then(res => res.json())
.then(data => console.log(data.firstName, data.email));

// 16.
function getFullName(userId) {
  return fetch(`https://dummyjson.com/users/${userId}`)
    .then(res => res.json())
    .then(data => `${data.firstName} ${data.lastName}`);
}

// 17.
fetch("https://dummyjson.com/users/1")
.then(res => res.text())
.then(console.log);

// 18.

```

```
fetch("https://dummyjson.com/users/1")
  .then(res => {
    res.json().then(console.log);
    res.json().then(console.log); // Erro: body já foi usado
  });
```

// 19.

```
fetch("https://dummyjson.com/products/1")
  .then(res => res.json())
  .then(prod => {
    console.log("Produto:", prod.title);
    return fetch("https://dummyjson.com/users/1");
  })
  .then(res => res.json())
  .then(user => console.log("Usuário:", user.firstName));
```

// 20.

```
fetch("https://dummyjson.com/erro")
  .then(res => res.json())
  .catch(console.error);
```

// ASYNC/AWAIT (21–30)

// 21.

```
async function fetchProduct() {
  const res = await fetch("https://dummyjson.com/products/1");
  const data = await res.json();
  console.log(data.title);
}
```

// 22.

```
async function getUsername(id) {
  const res = await fetch(`https://dummyjson.com/users/${id}`);
  const user = await res.json();
  return user.firstName;
}
```

// 23.


```

async function fetchWithError() {
  try {
    const res = await fetch("https://dummyjson.com/erro");
    const data = await res.json();
    console.log(data);
  } catch (error) {
    console.error("Erro:", error);
  }
}

// 24.
async function getProductPrice(id) {
  const res = await fetch(`https://dummyjson.com/products/${id}`);
  const data = await res.json();
  return data.price;
}

// 25.
async function getTitles() {
  const titles = [];
  for (let i = 1; i <= 3; i++) {
    const res = await fetch(`https://dummyjson.com/products/${i}`);
    const data = await res.json();
    titles.push(data.title);
  }
  return titles;
}

// 26.
async function fetchSequential() {
  for (let i = 1; i <= 3; i++) {
    const res = await fetch(`https://dummyjson.com/products/${i}`);
    const data = await res.json();
    console.log(data.title);
  }
}

// 27.

```

```
async function loadProduct(id) {  
  console.log("Carregando...");  
  const res = await fetch(`https://dummyjson.com/products/${id}`);  
  const data = await res.json();  
  console.log(data.title);  
}
```

// 28.

```
async function getUserAvatar(id) {  
  const res = await fetch(`https://dummyjson.com/users/${id}`);  
  const user = await res.json();  
  const img = document.createElement("img");  
  img.src = user.image;  
  document.body.appendChild(img);  
}
```

// 29.

```
async function fetchWithCustomError() {  
  const res = await fetch("https://dummyjson.com/products/1");  
  throw new Error("Erro proposital!");  
}
```

// 30.

```
async function getUserWithAgeCheck() {  
  try {  
    const res = await fetch("https://dummyjson.com/users/1");  
    const user = await res.json();  
    if (user.age < 30) throw new Error("Usuário muito jovem");  
    console.log(user.firstName);  
  } catch (err) {  
    console.error(err);  
  }  
}
```